

EN UNA FRASE Sacas del código los valores que cambian según el entorno: los defines en ficheros de propiedades, los inyectas con `@Value` o `@ConfigurationProperties` y cambias de configuración (y de nivel de log) con los perfiles `dev` y `prod`, sin recompilar.

LO ESENCIAL

<code>src/main/resources/application.properties</code>	Fichero central de configuración externalizada; Spring lo lee al arrancar
Propiedad de Spring Boot (<code>server.port</code> , <code>spring.application.name</code>)	Propiedad que el framework reconoce y procesa
Propiedad propia (<code>biblioteca.max-libros</code>)	La defines tú, con prefijo propio y en kebab-case
<code>@Value("\${prop}")</code>	Inyecta una propiedad suelta en un campo o parámetro de constructor
<code>@Value("\${prop:valor}")</code>	Valor por defecto si la propiedad no existe; sin él, Spring falla al arrancar
<code>@ConfigurationProperties(prefix = "biblioteca")</code>	Vincula todo un prefijo a una clase tipada con getters y setters; <code>max-libros</code> se enlaza con <code>maxLibros</code>
<code>spring-boot-configuration-processor</code>	Dependencia (<code>optional</code>) que habilita el autocompletado de tus propiedades en el IDE
Perfil de Spring	Nombre de un entorno; activa su fichero <code>application-{perfil}.properties</code> , que sobrescribe al base
<code>spring.profiles.active</code>	Indica qué perfil está activo (en el fichero base o por línea de comandos)
<code>@Profile("dev")</code>	El bean solo se crea con ese perfil activo; con otro perfil, Spring ignora la clase
<code>logging.level.paquete=NIVEL</code>	Nivel de log por paquete: <code>TRACE</code> , <code>DEBUG</code> , <code>INFO</code> , <code>WARN</code> , <code>ERROR</code> , <code>OFF</code>
SLF4J (Logger, LoggerFactory)	API de logging: <code>log.debug(...)</code> , <code>log.info(...)</code> ; el nivel se decide en las propiedades, no en el código

ERRORES TÍPICOS

❶ **Could not resolve placeholder 'biblioteca.telefono' y la aplicación no arranca**

✓ la propiedad de ese `@Value` no existe: añádela al fichero o pon un valor por defecto, `@Value("${biblioteca.telefono:Sin teléfono}")`.

❷ **Los campos de `BibliotecaConfig` salen a null o 0 sin ningún error**

✓ faltan getters y setters; Spring no puede escribir en campos privados sin setter.

❸ **El perfil aparece como activo pero los valores no cambian**

✓ `application-{perfil}.properties` no existe o su nombre no coincide con el perfil; Spring arranca igual y no avisa. Comprueba el fichero en `src/main/resources/`.

❹ **La configuración se carga de forma inesperada**

✓ conviven `application.properties` y `application.yml`. Elige un formato; en este volumen, `.properties`.

❺ **Cambias `logging.level...` y el log sigue igual**

✓ el nombre del paquete está mal escrito (por ejemplo, `servicios` en vez de `servicio`); la propiedad se ignora en silencio.

EN EL PROYECTO BIBLIOTECA

Nacen `configuracion/BibliotecaConfig` (propiedades `biblioteca.*`), `controlador/InfoController` con `GET /info` y `configuracion/CargadorDatosPrueba`, que carga los tres libros canónicos solo con el perfil `dev`. `BibliotecaService` gana un logger SLF4J. `application.properties` queda como base común con el perfil activo, y aparecen `application-dev.properties` y `application-prod.properties`.

Configuración, propiedades y perfiles

■ CÓDIGO MÍNIMO Y PETICIONES

Los ficheros de propiedades del proyecto: el base activa el perfil y el de perfil pone lo específico del entorno.

```
# application.properties
spring.application.name=biblioteca-spring
spring.profiles.active=dev

# application-dev.properties
server.port=8080
logging.level.com.javaesencial.biblioteca=DEBUG
biblioteca.nombre=Biblioteca Municipal "El Quijote"
biblioteca.version=2.0
biblioteca.max-libros=500
biblioteca.moda-mantenimiento=false

# application-prod.properties (lo que cambia)
server.port=${PORT:8080}
logging.level.com.javaesencial.biblioteca=WARN
```

PROPERTIES

La clase tipada que recibe el prefijo `biblioteca` (recortada: cada campo necesita su getter y su setter).

```
package com.javaesencial.biblioteca.configuracion;

@Component
@ConfigurationProperties(prefix = "biblioteca")
public class BibliotecaConfig {

    private String nombre;
    private String version;
    private int maxLibros;
    private boolean modaMantenimiento;

    public int getMaxLibros() { return maxLibros; }
    public void setMaxLibros(int maxLibros) { this.maxLibros = maxLibros; }
    // ... getters y setters del resto de campos
}
```

JAVA

```
# Activar un perfil al ejecutar el JAR (recomendado en despliegues)
java -jar biblioteca-spring.jar --spring.profiles.active=prod

# Activar un perfil con Maven durante el desarrollo
mvn spring-boot:run -Dspring-boot.run.profiles=dev

# El log confirma el perfil: The following 1 profile is active: "dev"
# Ver la configuración cargada: GET http://localhost:8080/info
```

TERMINAL